

# Contest Platform with Live Leaderboard

Database Implementation & Core Functionalities (MongoDB + Redis)

|                   |                                                                                  |
|-------------------|----------------------------------------------------------------------------------|
| <b>Team</b>       | Upayan Mazumder (24BDS0367), Aditya Rawat (24BCE2992), Shloak Sinha (24BDS0378)  |
| <b>Stack</b>      | MongoDB 7 (single-node replica set <code>rs0</code> ) + Redis 7, FastAPI backend |
| <b>Repository</b> | github.com/upayanmazumder/rankstack                                              |

## 1. Database implementation

| Collection  | Key fields                                                                |
|-------------|---------------------------------------------------------------------------|
| users       | name, email, passwordHash, role, totalScore                               |
| teams       | name, memberIds[], totalScore                                             |
| contests    | title, startTime, endTime, status, problemIds[], participants[]           |
| problems    | contestId, type (mcq/coding/subjective), difficulty, points, attemptCount |
| submissions | contestId, problemId, submittedBy, status, score                          |

Every collection is created with a `$jsonSchema` validator, so MongoDB rejects a bad document at insert time instead of leaving the check to the API. `scripts/init_db.py` creates the collections, validators and indexes, and can be re-run safely.

Relationships:

- Contest → Problems
- Contest → Participants (users or teams)
- User/Team → Submissions
- Team → Users
- Submission → Problem

Sample data loaded by `scripts/seed.py`:

| Collection   | Records   |
|--------------|-----------|
| users        | 15        |
| teams        | 5         |
| contests     | 5         |
| problems     | 15        |
| submissions  | 48        |
| <b>Total</b> | <b>88</b> |

The rubric asks for 50+ records. The seed gives 88, and the references between them are consistent (every submission points at a real problem, contest and participant).

## 2. CRUD operations

33 REST endpoints across the five collections plus sessions, all reachable from Swagger UI at `/docs`.

| Operation | Implemented for                                     |
|-----------|-----------------------------------------------------|
| Create    | Users, teams, contests, problems, submissions       |
| Read      | All 5 collections (single record and filtered list) |

|        |                                               |
|--------|-----------------------------------------------|
| Update | Users, teams, contests, problems, submissions |
| Delete | Users, teams, contests, problems, submissions |

```

POST /users {"name":"Doc User","email":"doc@rankstack.io","password":"secret123"}
-> 201 {"id":"6ab3f9a5...", "name":"Doc User", "role":"participant", "totalScore":0}

GET /users/6ab3f9a5... -> 200 {"id":"6ab3f9a5...", "name":"Doc User", ...}
PATCH /users/6ab3f9a5... -> 200 {"name":"Doc User (edited)", ...}
DELETE /users/6ab3f9a5... -> 204

```

## 3. Advanced NoSQL features

### 3.1 Aggregation pipelines

| Pipeline                 | What it answers                                   | Endpoint                                   |
|--------------------------|---------------------------------------------------|--------------------------------------------|
| Contest leaderboard      | total score and rank per contestant               | /aggregations/leaderboard/{id}             |
| Problem statistics       | average score per difficulty level                | /aggregations/avg-score-by-difficulty      |
| Submission statistics    | submissions by status, per contest                | /aggregations/submission-status-by-contest |
| Problems solved per user | distinct correct problems across contests         | /aggregations/problems-solved-per-user     |
| Team performance         | \$lookup from teams to their members' submissions | /aggregations/team-performance             |

```

Sample output: {"name": "Team Lavender Trial", "totalScore": 195.0, "submissionCount": 12,
"memberCount": 3}

```

### 3.2 Indexing

| Index                    | Collection and keys                                  |
|--------------------------|------------------------------------------------------|
| uniq_email               | users (email), unique                                |
| by_contest               | problems (contestId)                                 |
| by_contest_and_submitter | submissions (contestId, submittedBy.refId), compound |
| by_problem               | submissions (problemId)                              |
| by_member                | teams (memberIds), multikey                          |
| by_status                | contests (status)                                    |

The demo script runs `explain()` on two of these queries and prints the winning plan, which comes back as `IXSCAN` with the index name, so the query is not scanning the whole collection.

### 3.3 Transactions

Four operations in `api/services.py` run inside a session transaction, which is why Mongo runs as a replica set:

- `create_submission`: insert the submission and increment `problem.attemptCount`
- `update_submission_status`: set status and score, then add the score to the user or team total
- `add_team_member` and `remove_team_member`: update the team and the user's back-reference together

### 3.4 Redis

- Live leaderboard per contest in a sorted set ( `ZINCRBY` on score change, `ZREVRANGE` to read the top N)
- Login sessions as keys with a TTL of 3600s
- Submission rate limiting: a counter per user in a 60s window, capped at 10

## 4. Intermediate demonstration

`make demo` runs `scripts/demo.py` against the seeded database and prints each step:

1. Connection check on Mongo and Redis
2. CRUD on a user: create, read, update, delete
3. Indexed query on `problems.contestId` with the winning plan and index name
4. Compound-indexed query on submissions by contest and submitter
5. Transaction: submission inserted while `attemptCount` and the user total move with it
6. All five aggregation pipelines with their results
7. Redis leaderboard, session TTL and rate-limit counter

*The demo cleans up what it creates, so it can be run repeatedly during the review.*

## 5. Checklist

| Criterion                  | Marks | Where                                                                                         | Status |
|----------------------------|-------|-----------------------------------------------------------------------------------------------|--------|
| Database implementation    | 2     | Section 1, <code>scripts/init_db.py</code>                                                    | Done   |
| CRUD operations            | 3     | Section 2, <code>api/routes/</code>                                                           | Done   |
| Advanced NoSQL features    | 5     | Section 3, <code>api/aggregations/</code> , <code>api/services.py</code>                      | Done   |
| 50+ sample records         | –     | 88 records from <code>scripts/seed.py</code>                                                  | Done   |
| Intermediate demonstration | –     | <code>scripts/demo.py</code>                                                                  | Done   |
| Source code repository     | –     | <a href="https://github.com/upayanmazumder/rankstack">github.com/upayanmazumder/rankstack</a> | Done   |

## 6. Running it

```
make up          # MongoDB replica set + Redis in Docker
make init-db     # collections, $jsonSchema validators, indexes
make seed        # 88 sample records
make demo        # the walkthrough in section 4
make api         # FastAPI on http://localhost:8000/docs
```